Experiment 4
Block Matrix Multiplication

Kaushik Balaji M S *M.Tech in System-on-chip Design*
Indian Institute of Technology, Palakkad
152502010@smail.iitpkd.ac.in

I. Introduction

In this experiment, Multiplication of Matrices is being done using Block or Tile partitioning, where single matrix gets broken down into several chunks or tiles, which then gets executed in parallel, for better performance.

$$C_{ij} = \sum_{k=0}^{M-1} A_{ik} \cdot B_{kj} \tag{1}$$

Here,

$$\text{Total Operations} = N \times P \times M \tag{2}$$

and for $N = 16$, Total Operations $= 16^3 = 4096$

II. Matrix Multiplication models used

```
#define N 16
#define M 16
#define P 16
typedef ap_axiu<512,0,0,0> axis_512_t;

    int A[N][M];
    int B[M][P];
    int C[N][P];

    // Load Matrix A
load_A:
    for(int i=0;i<N;i++){
        axis_512_t pkt = s_axis.read();
        for(int k=0;k<M;k++){
#pragma HLS UNROLL
            A[i][k] = (int)pkt.data.range(k*32+31, k*32);
        }
    }

    // Load Matrix B
load_B:
    for(int i=0;i<M;i++){
        axis_512_t pkt = s_axis.read();
        for(int k=0;k<P;k++){
#pragma HLS UNROLL
            B[i][k] = (int)pkt.data.range(k*32+31, k*32);
        }
    }

    // Write result matrix
store:
    for(int i=0;i<N;i++){

```

```
        ap_uint<512> result = 0;

        for(int j=0;j<P;j++){
#pragma HLS UNROLL
            result.range(j*32+31, j*32) = (ap_uint<32>)C[i][j];
        }

        axis_512_t out_pkt;
        out_pkt.data = result;
        out_pkt.keep = -1;
        out_pkt.last = (i == N-1);

        m_axis.write(out_pkt);
    }
```

Listing 1. Input Read and Write from and into AXI Stream

The code snippet in Listing: 1 shows how Input matrices are read from AXI Stream and output matrix written to AXI Stream using the ***hls::stream***. This is the same for all the Matrix multiplication models implemented below.

```

#pragma HLS INTERFACE axis port=s_axis
#pragma HLS INTERFACE axis port=m_axis
#pragma HLS INTERFACE ap_ctrl_none port=return

    // NAIVE computation
row:
    for(int i=0;i<N;i++){
    col:
        for(int j=0;j<P;j++){

            int sum = 0;

        product:
            for(int k=0;k<M;k++){
                sum += A[i][k] * B[k][j];
            }

            C[i][j] = sum;
        }
    }
```

Listing 2. Naive Matrix Multiplication without any Pipeline

Here the Multiplication is done in sequential, without any Pipeline usage.

```
// Pipelined computation
row:
    for(int i=0;i<N;i++){
    col:
        for(int j=0;j<P;j++){

#pragma HLS PIPELINE II=1

            int sum = 0;

        product:
            for(int k=0;k<M;k++){
                sum += A[i][k] * B[k][j];
            }

            C[i][j] = sum;
        }
    }
```

Listing 3. Basic Pipelined Matrix Multiplication

Here the column loop is pipelined, allowing overlapping execution of successive iterations while the inner product loop performs the accumulation. This reduces the Latency compared to sequential multiplication.

```
row:
    for(int i = 0; i < N; ++i) {
    col:
        for(int j = 0; j < P; ++j) {
        #pragma HLS PIPELINE II=4
            int acc = 0;
        product:
            for(int k = 0; k < M; ++k) {
                #pragma HLS UNROLL
                acc += A[i][k] * B[k][j];
            }
            AB[i][j] = acc;
        }
    }

```

Listing 4. Loop unrolling with the Column loop pipelined

When the pipeline directive is applied to the column loop, the pipeline only overlaps the column computations within a single row, while the outer row loop executes sequentially. As a result, the rows cannot overlap in execution, which significantly increases the total latency compared to pipelining the row loop.

```
row:
    for(int i = 0; i < N; ++i) {
        #pragma HLS PIPELINE II=4
    col:
        for(int j = 0; j < P; ++j) {
            int acc = 0;
        product:
            for(int k = 0; k < M; ++k) {
                #pragma HLS UNROLL
                acc += A[i][k] * B[k][j];
            }
            AB[i][j] = acc;
        }
    }
```

Listing 5. Loop unrolling with Row Loop pipelined

III. Performance Analysis

Implementation	**Latency (cycles)**	**Latency (ns)**	**DSP**	**FF**	**LUT**
Naive Matrix Multiply	2231	2.231e4	6	2482	2939
Pipelined Matrix Multiplication	336	3.3603e3	48	14354	6374
AXI Stream + Unrolled Product Loop	134	1.34e3	192	32898	16821
Pipeline on Column Loop (Inefficient Placement)	1102	1.10e4	12	20706	6721

TABLE I
COMPARISON OF DIFFERENT MATRIX MULTIPLICATION IMPLEMENTATIONS SYNTHESIZED USING VITIS HLS

The naive implementation performs the matrix multiplication sequentially, resulting in the highest latency of 2231 cycles. In this design, the inner product loop executes serially and no parallel hardware resources are utilized.

By applying loop pipelining to the computation loops (Listing 3), the pipelined implementation significantly reduces the latency to 336 cycles. The pipeline allows overlapping execution of successive loop iterations, improving throughput while moderately increasing resource usage.

Further optimization was achieved by completely unrolling the product loop and using wide AXI streaming interfaces. Loop unrolling allows all multiplication operations of the dot product to execute in parallel using dedicated DSP units, while the 512-bit AXI stream transfers an entire row of matrix elements in a single cycle.

When the pipeline directive is applied to the row loop as shown in Listing 5, the execution of multiple rows overlaps. Since the product loop is fully unrolled, the dot product for each matrix element is computed using parallel multipliers. This allows the accelerator to process multiple rows in a pipelined fashion, significantly reducing the total latency. The approximate computation time can be expressed as:

$$\text{Matrix read} + \text{write} + (\text{Number of rows} \times \text{Pipeline II}) \approx 134 \text{ cycles}$$

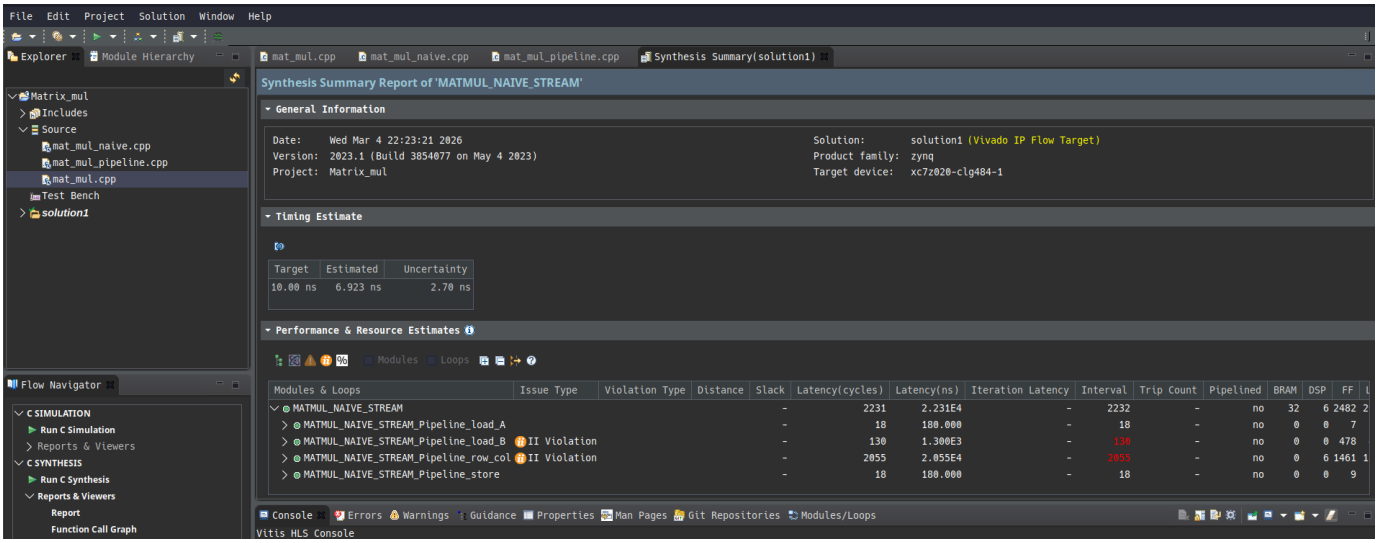

Fig. 1. Basic Matrix Multiplication

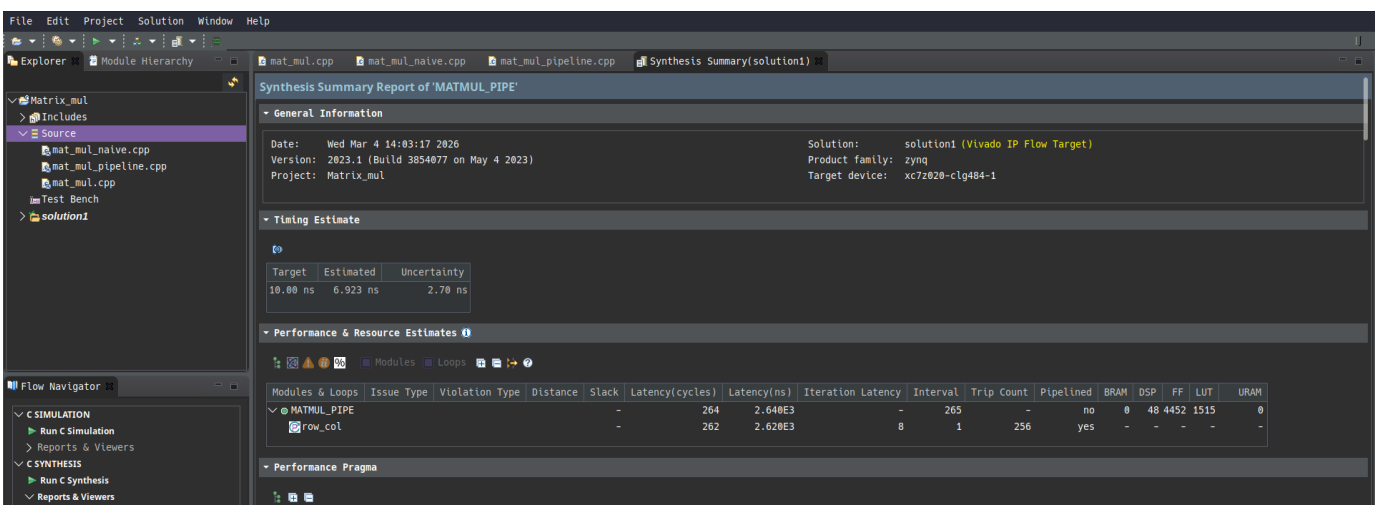

Fig. 2. Basic Pipelined Multiplication

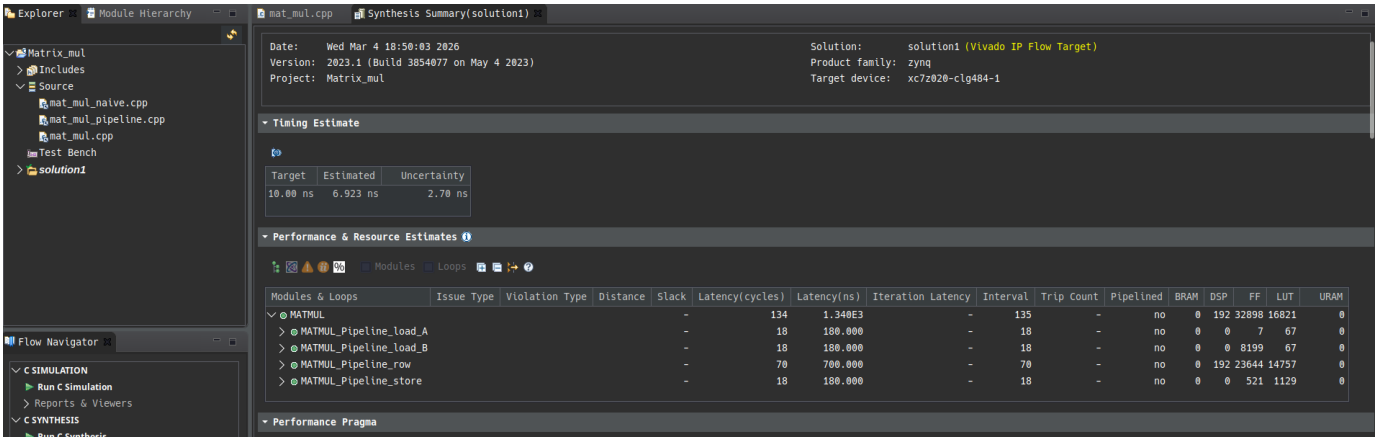

Fig. 3. Optimized (Loop unrolling + Pipelined) Matrix Multiplication

IV. Vivado Hardware creation

Once the IP was exported from Vitis HLS, that IP is used to make the platform for the zedboard. The components used are

1) Zynq 7000 Processing system
2) AXI DMA
3) Matrix multiplication IP from Vitis HLS

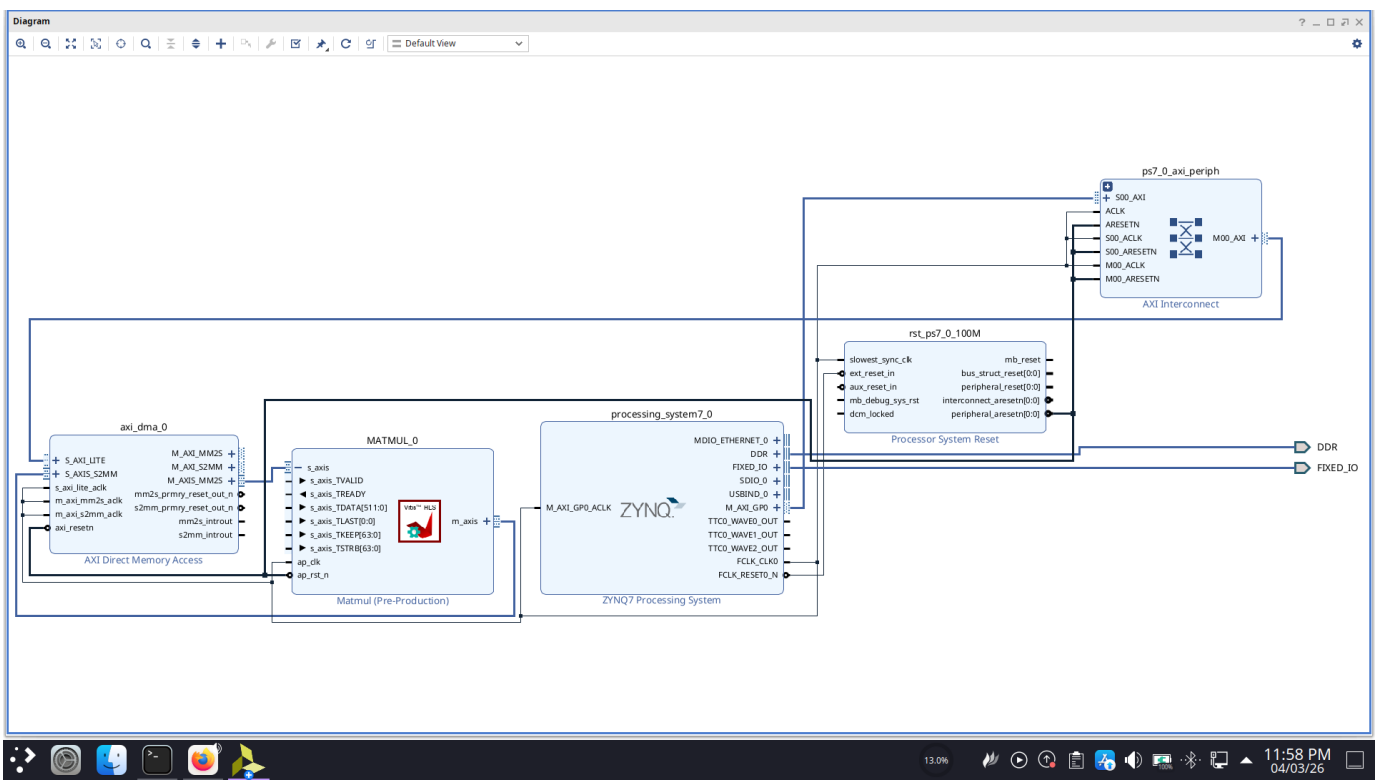

Fig. 4. Matrix Multiplier Block diagram

Once the block design is created, create a **HDL wrapper** for that design and proceed to synthesis, Implementation, and Bitstream Generation.

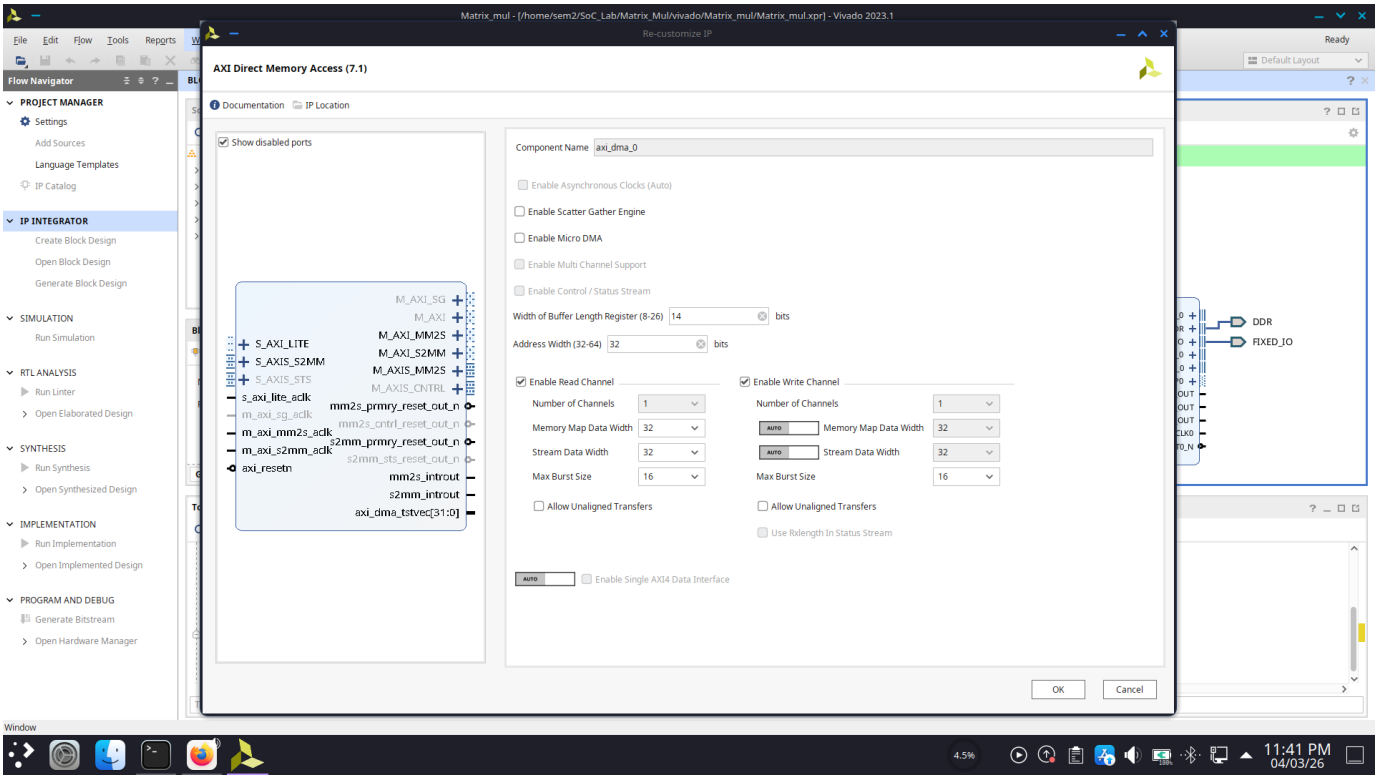

Fig. 5. Configuration for the AXI DMA

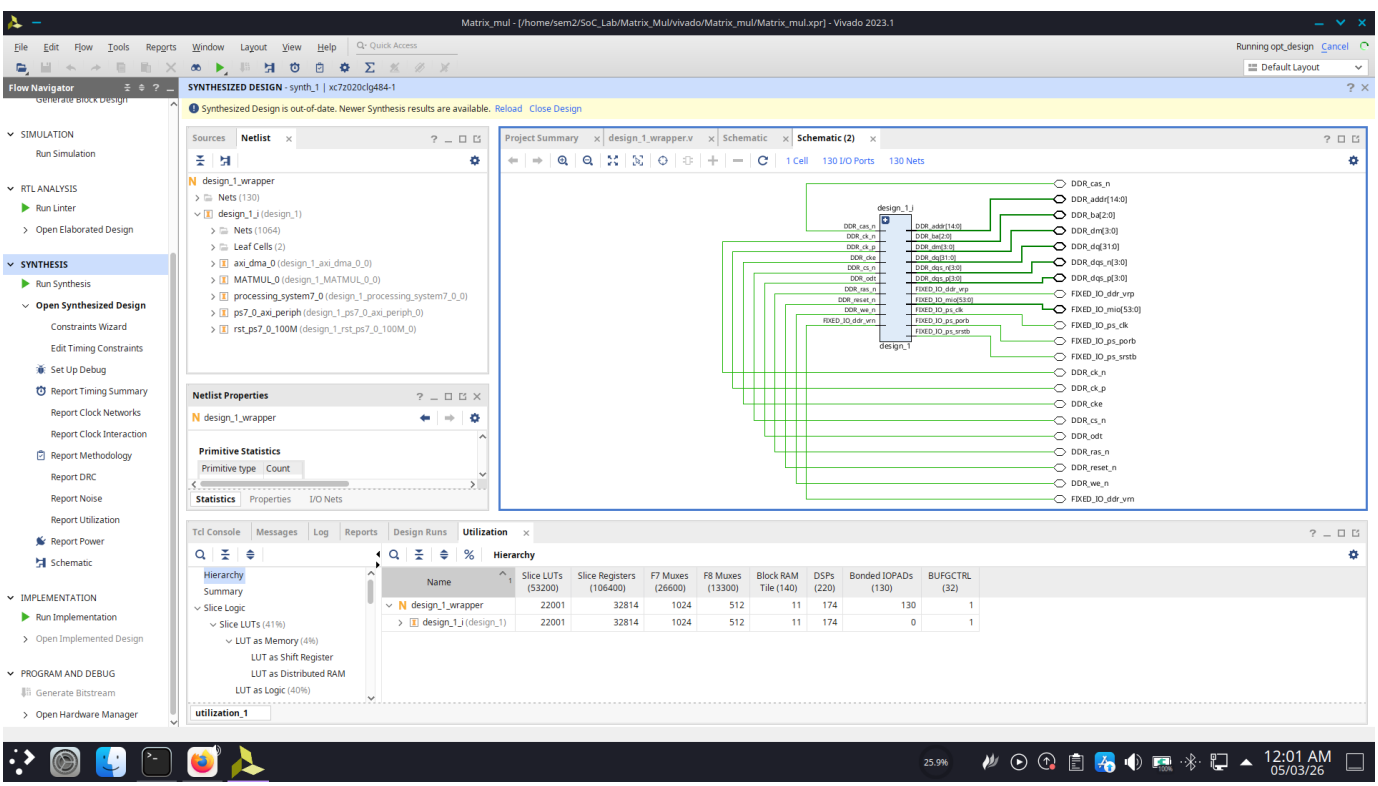

Fig. 6. Schematic and the Utilization report of the synthesized design

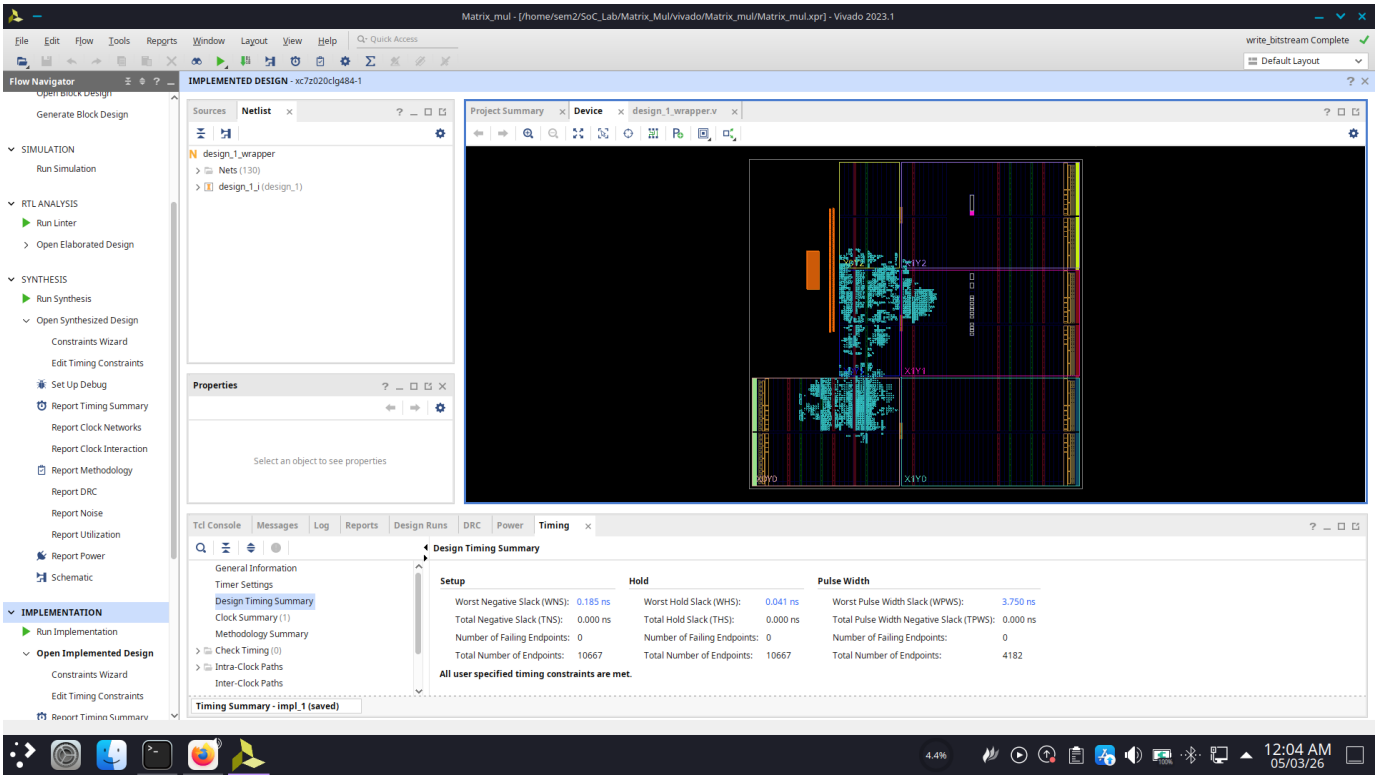

Fig. 7. Final design after implementation and Bitstream generation

The figure 7 gives the slack time for the final implemented design. Once all these are done, the design is exported, along with the generated bitstream.

V. Conclusion

In this experiment, matrix multiplication was implemented using Vitis HLS and optimized using various HLS directives such as pipelining, loop unrolling, and array partitioning. The results showed that applying these optimizations significantly reduced the overall latency compared to the naive implementation. However, the reduction in latency was achieved at the cost of increased hardware resource utilization. The optimized design was exported as a custom IP and successfully integrated into a Zynq-based hardware platform using Vivado with AXI DMA and AXI Stream interfaces. This experiment demonstrated how computationally intensive operations can be accelerated on FPGA hardware using HLS-based design techniques.